

SOFTWARE ENGINEERING

Prof. Dr. Christoph Andriessens
christoph.andriessens@hs-weingarten.de



ARCHITEKTURMUSTER

Wiederverwendbare Muster und Stile für Softwarearchitektur

Themen in dieser Vorlesung



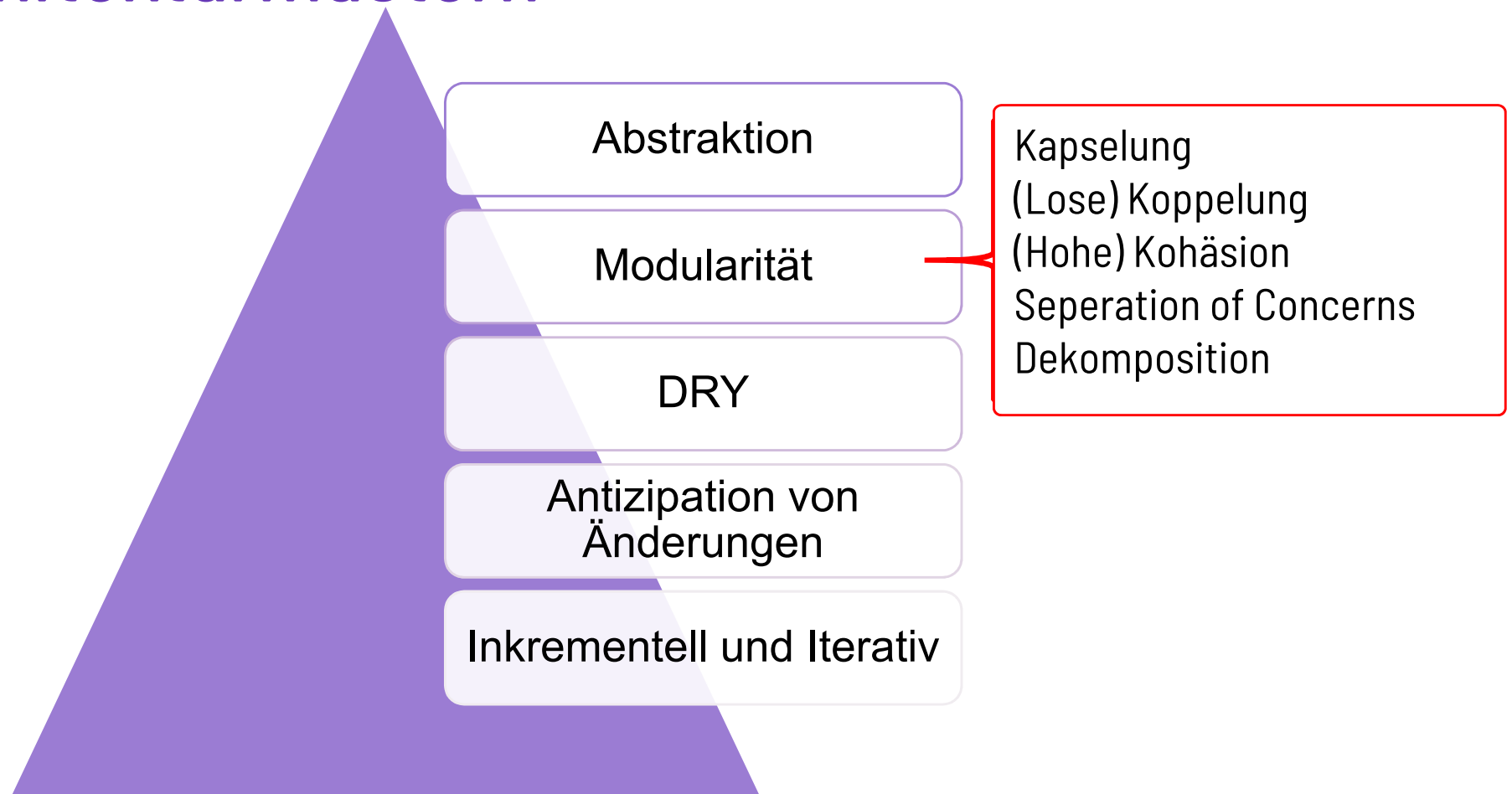
Architekturmuster

1. Client-Server-Architektur
2. Mehrschichten-Architektur
3. Model-View-Controller (MVC)-Architektur
4. Repositoriums-Architektur
5. Pipes-und-Filter-Architektur (Datenfluss-Architektur)

Architekturmuster

- Muster sind ein Mittel, Wissen **darzustellen**, zu **teilen** und **wiederverwenden**.
- Ein Architekturmuster ist eine stilisierte Beschreibung einer **bewährten Architektur-Praxis**, die in verschiedenen Umgebungen **ausprobiert** und **getestet** wurde.
- Muster sollen Informationen dazu enthalten, wann sie **nützlich** sind und **wann nicht**.
- Muster können in Tabellen oder graphischen Beschreibungen ausgedrückt werden.

Software Engineering-Prinzipien dienen auch der Strukturierung und Bewertung von Architekturmustern



Client-Server-Architektur 1

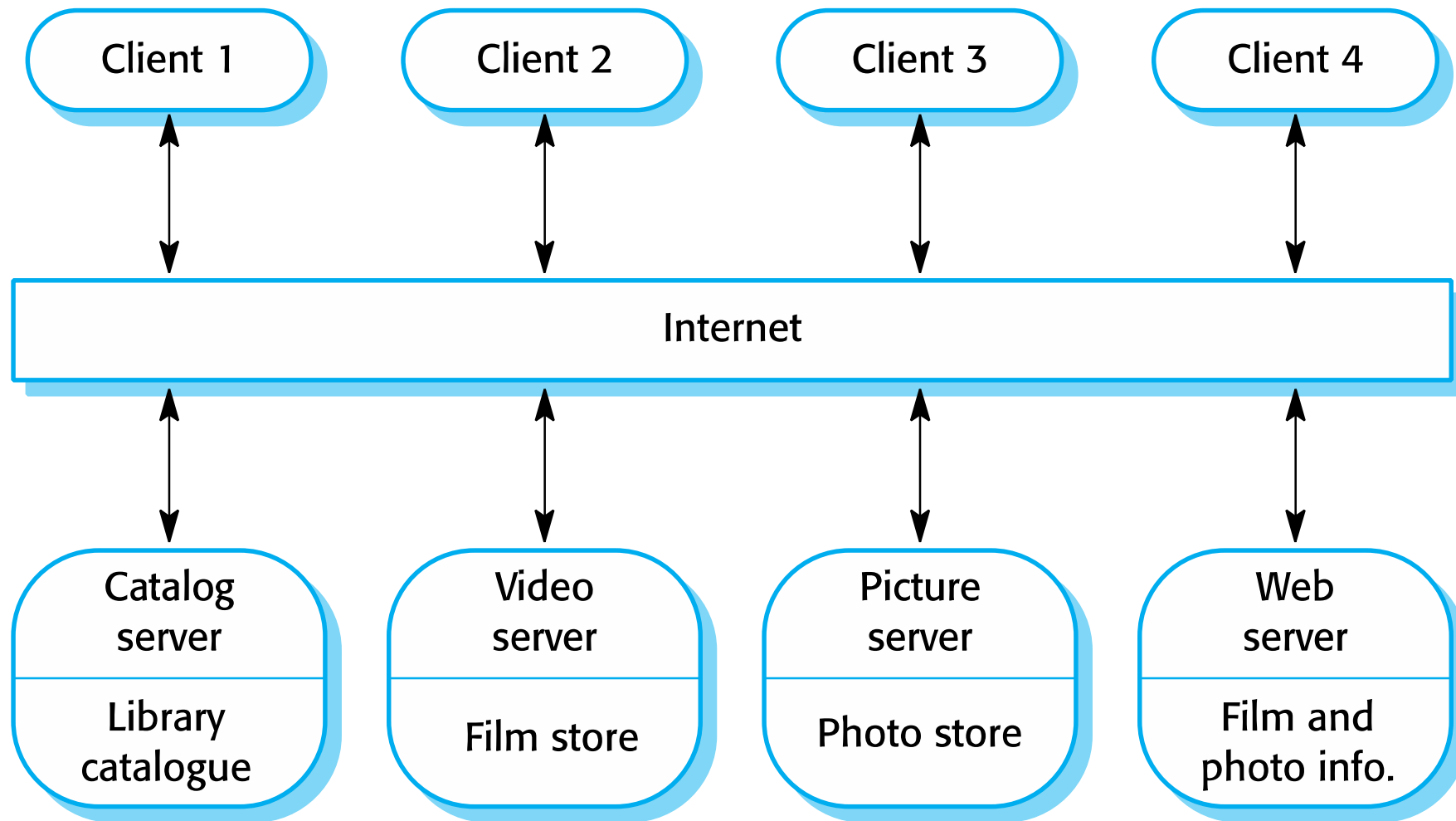
- Server: „Dienstgeber“ oder „Dienstleister“, Client: „Dienstnutzer“
- **Modell für verteilte Systeme**, dass Daten und ihre Verarbeitung über mehrere Komponenten verteilt
- Kann aber auch auf einem einzelnen Computer implementiert werden!

Client-Server-Architektur 2

Die Funktionalität des Systems wird bei Client-Server-Architektur in **Dienste** aufgeteilt.

- Eine Menge von **selbständigen Servern** bieten **Dienste** (Services) an wie Drucken, Daten Management etc.
 - Pro Server sind mehrere Dienste möglich
- Pro Server **rufen** eine Menge von **Clients** diese Dienste auf
- **Netzwerk** verbindet Clients mit diesen Servern

Client-Server-Architektur für eine Film/Video/DVD-Bibliothek



Unterschiedliche Sichtweisen in der Architektur am Beispiel Client/Server

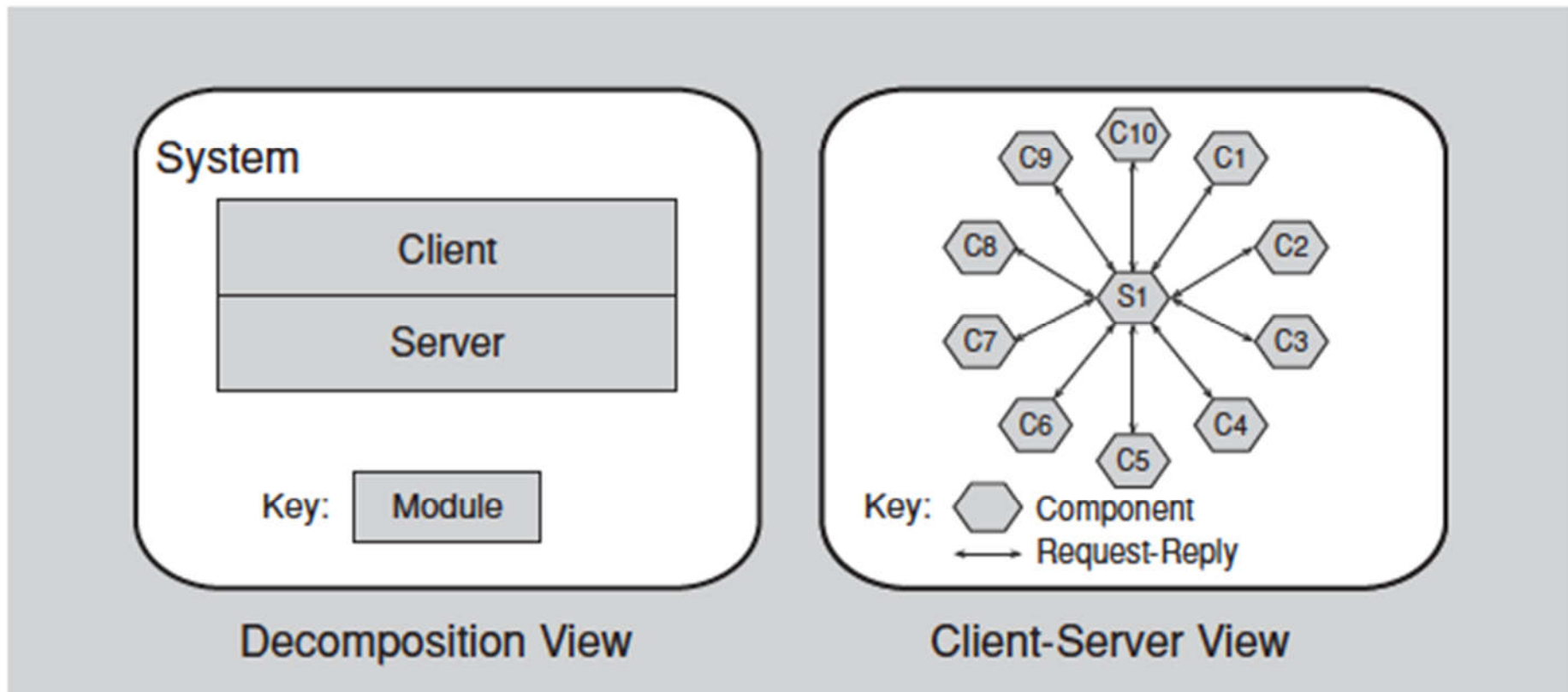


FIGURE 1.2 Two views of a client-server system

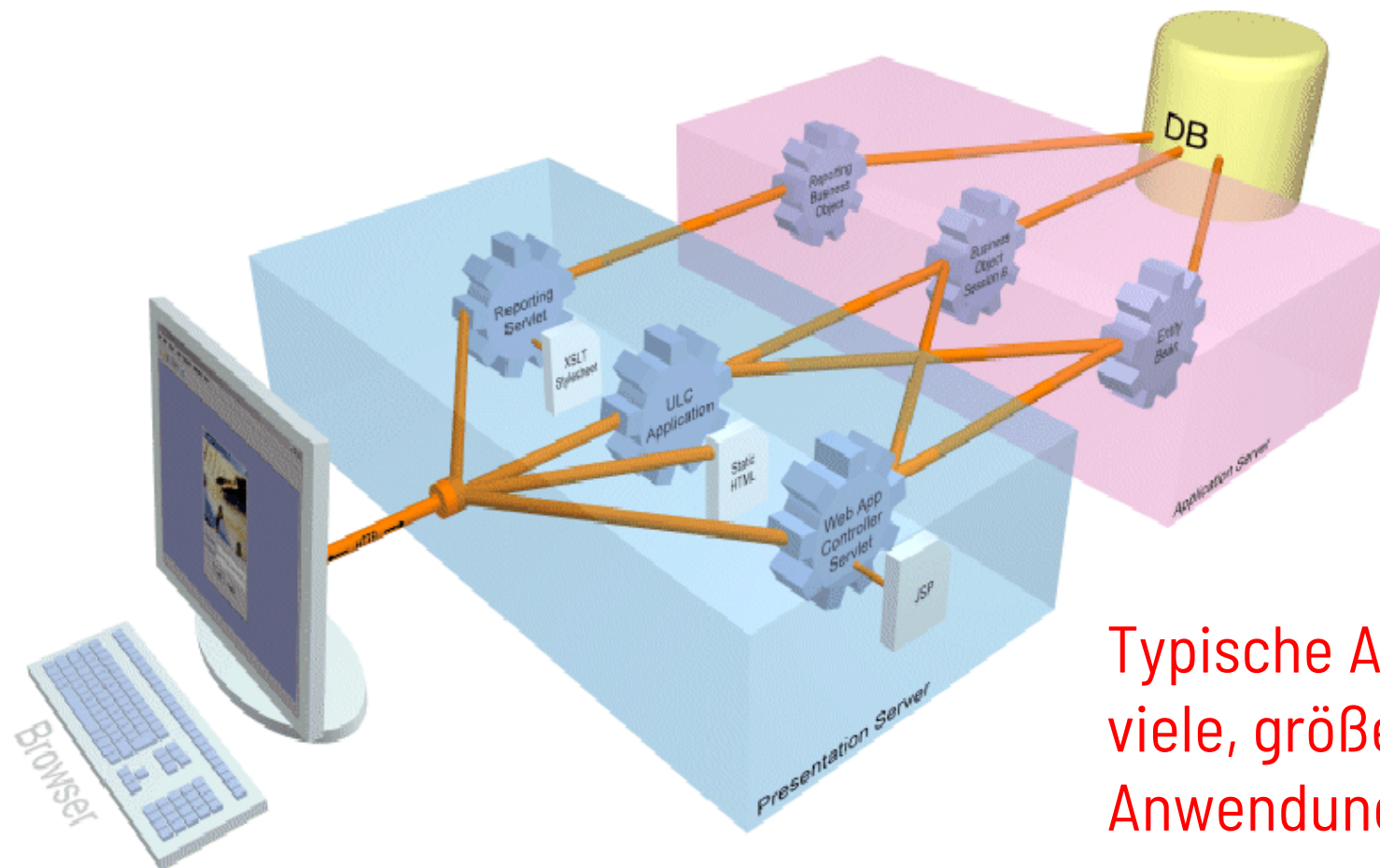
© Len Bass, Paul Clements, Rick Kazman, distributed under Creative Commons Attribution License

Client-Server-Architektur

Name	Client-Server
Einsatz	Wenn Daten in einer gemeinsamen Datenbank liegen, auf die von unterschiedlichen Orten zugegriffen wird. Außerdem wenn die Systemlast variabel ist, da Server repliziert werden können.
Vorteile	Hauptvorteil ist, dass die Server über ein Netzwerk verteilt werden können. Außerdem Bündelung : Allgemeine Funktionalität (etwa Druckservice) kann dabei allen Clients gemeinsam zur Verfügung gestellt werden ohne mehrfach implementiert werden zu müssen.
Nachteile	Jeder Dienst ist ausfallkritisch (single point of failure) und so anfällig für Denial of Service-Angriffe oder Ausfall. Performance kann unvorhersagbar sein, da sie vom Netzwerk wie von Servern abhängt. Vielleicht zusätzliche Management-Probleme , wenn Server zu unterschiedlichen Organisationen gehören.

Mehrschichtenarchitektur („Layered Architecture“)

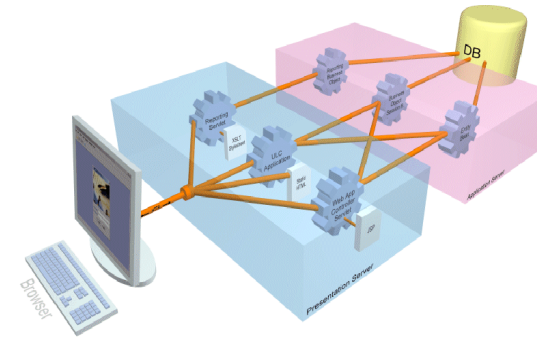
Prinzip im Diagramm



Typische Architektur für
viele, größere
Anwendungen

Mehrschichtenarchitektur

Charakteristika



- **Strukturierung** des Systems in logische Blöcke („Schichten“), die Subsysteme, Komponenten oder Module enthalten
- **Positionierung** sämtlicher Anforderungen in den Schichten
- **„Ownership“** von Funktionalität und Daten in einer Schicht
- **Kollaboration** zwischen Schichten
- **Hierarchisch**: Eine Schicht kennt *nur* die Schichten, die sie selber benutzt („Zusammenarbeit nur von oben nach unten“)

Verwendete SE-Prinzipien:

- Lose Kopplung
 - Minimale Abhängigkeit zwischen Komponenten
- Hohe Kohäsion
 - Bereiche mit zusammengehörenden Inhalten und Funktionalitäten, mit gleichen oder ähnlichen Datenanforderungen
- Redundanzvermeidung/-minimierung

Mehrschichtenarchitektur

Schematische Darstellung



Hierarchie: Zugriff nur von „oben“ nach „unten“!

Mehrschichtenarchitektur

Zwischenfazit

- Zerteilt das System in eine Menge von **Schichten** auf, die Dienste anbieten
- Wird eingesetzt, um die **Schnittstellen** von Subsystemen zu modellieren
- Unterstützt **inkrementelle Entwicklung** von Subsystemen in unterschiedlichen Schichten. Ändert sich die Schnittstelle einer Schicht, sind nur Nachbarschichten betroffen
- Es **kann** aber auch „**künstlich**“ sein, Systeme so zu strukturieren

Mehrschichtenarchitektur

Eine generische 4-Schichten-Architektur

User interface

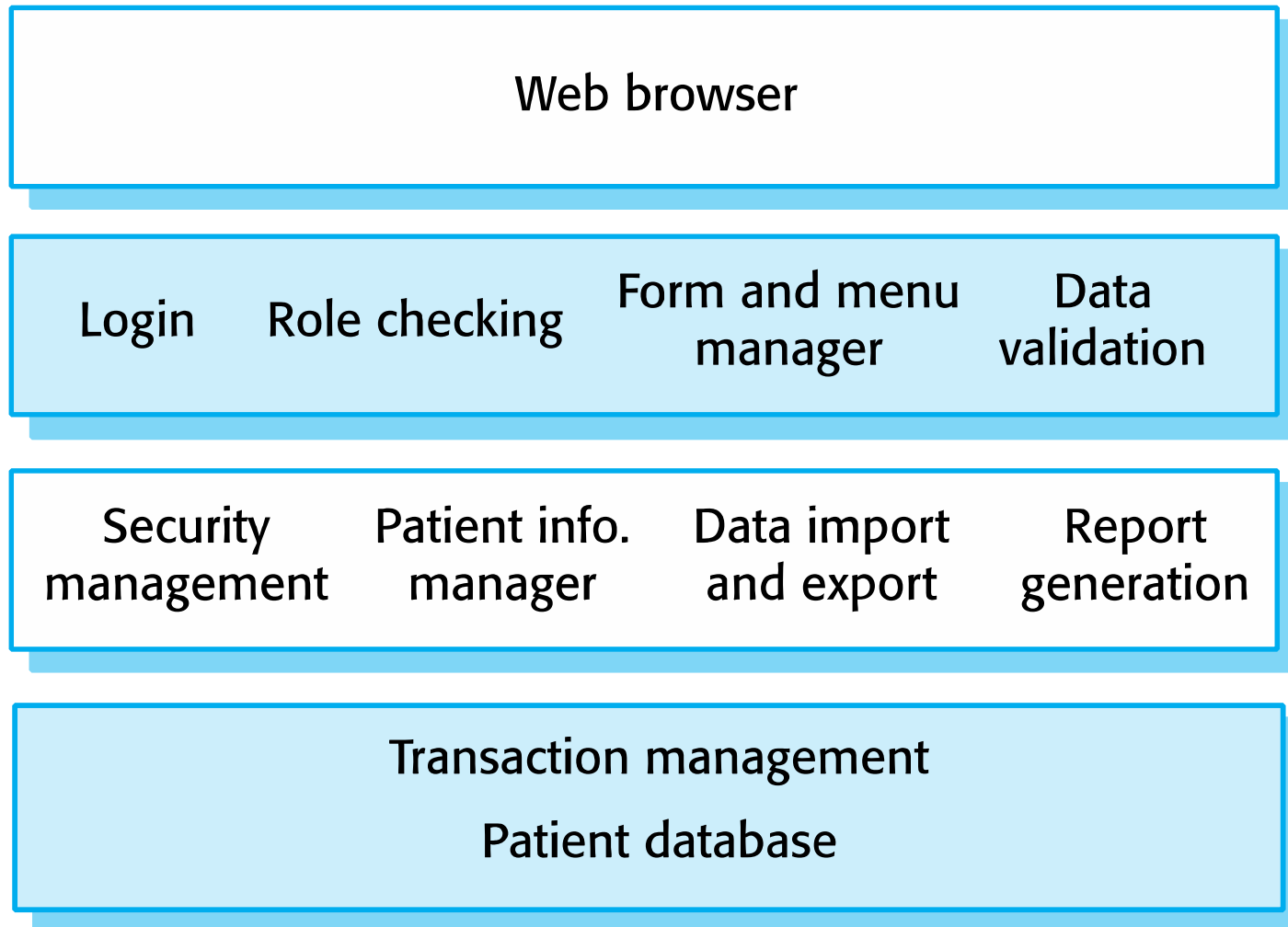
User interface management
Authentication and authorization

Core business logic/application functionality
System utilities

System support (OS, database etc.)

Mehrschichtenarchitektur: Beispiel

Architektur des medizinischen Informationssystems



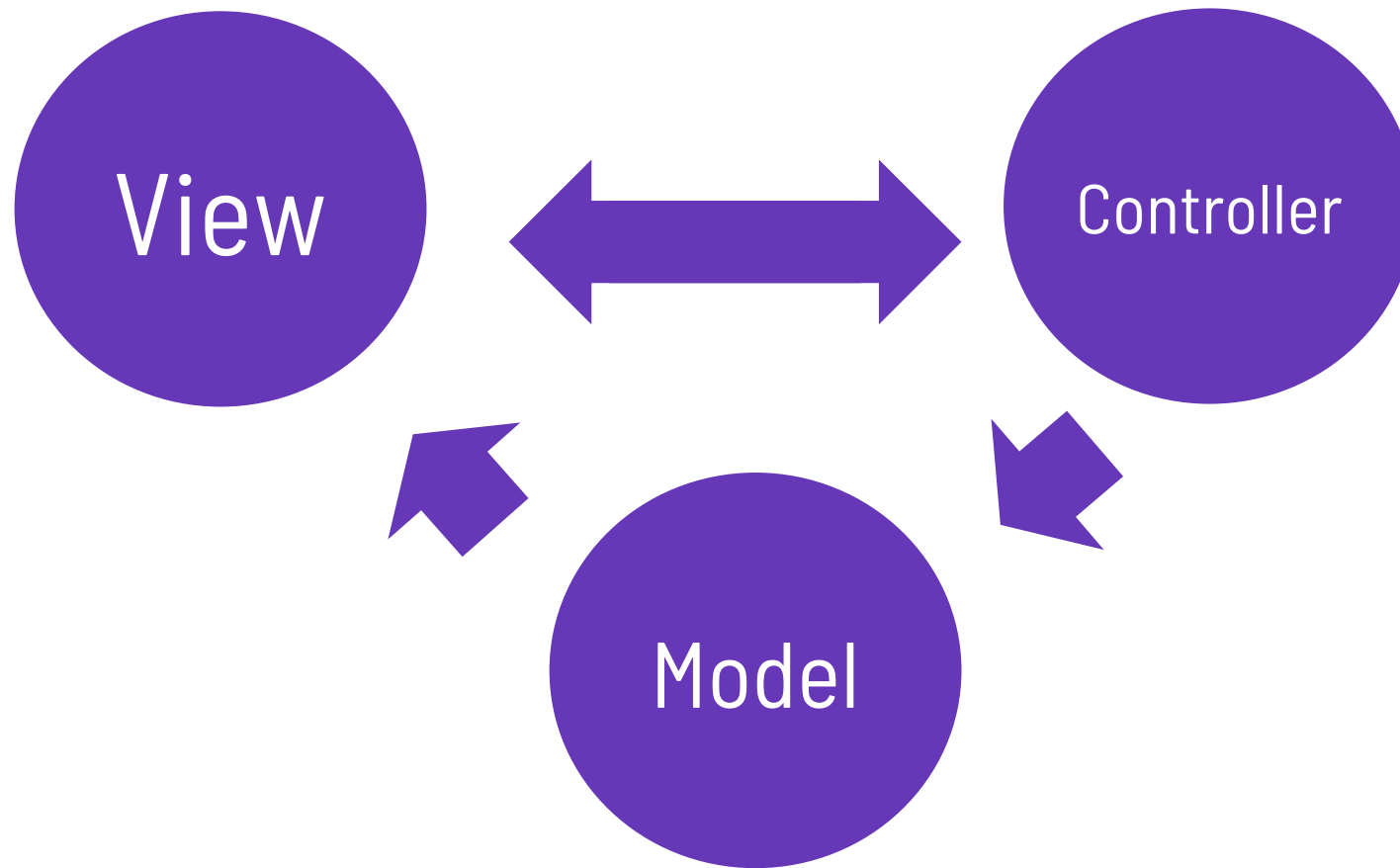
Mehrschichtenarchitektur

Name	Mehrschichtenarchitektur
Einsatz	Wenn die Entwicklung über mehrere Teams verteilt ist und jedes Team für eine Funktions-Schicht verantwortlich ist; wenn mehrstufige Sicherheit eine Anforderung ist; Aufbau neuer Funktionen auf existierenden Systemen
Vorteile	Erlaubt Ersatz ganzer Schichten , solange die Schnittstelle beibehalten wird. Zuverlässigkeit des Systems kann durch redundante Funktionen zwischen verschiedenen Schichten gesteigert werden.
Nachteile	In der Praxis ist eine klare Aufteilung zwischen den Schichten nicht leicht. Manchmal muss eine "höhere" Schicht direkt mit "niedrigeren" interagieren ohne korrekt über die Schichten dazwischen zu gehen. Performance kann durch Anforderungsverarbeitung auf mehreren Ebenen ein Problem werden.

Architekturmuster

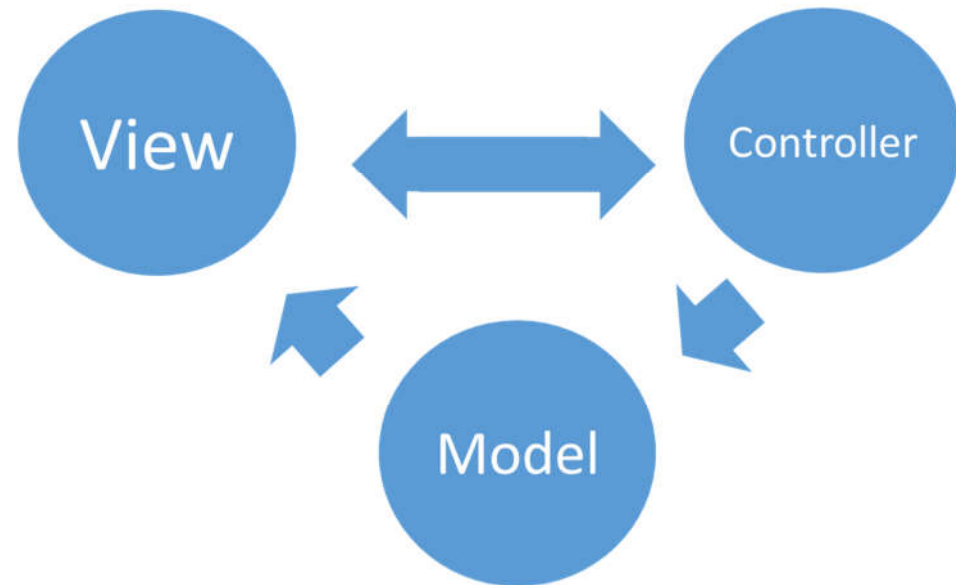
1. Client-Server-Architektur
2. Mehrschichten-Architektur
3. Model-View-Controller (MVC)-Architektur
4. Repositoriums-Architektur
5. Pipes-und-Filter-Architektur (Datenfluss-Architektur)

Model-View-Controller-Architektur



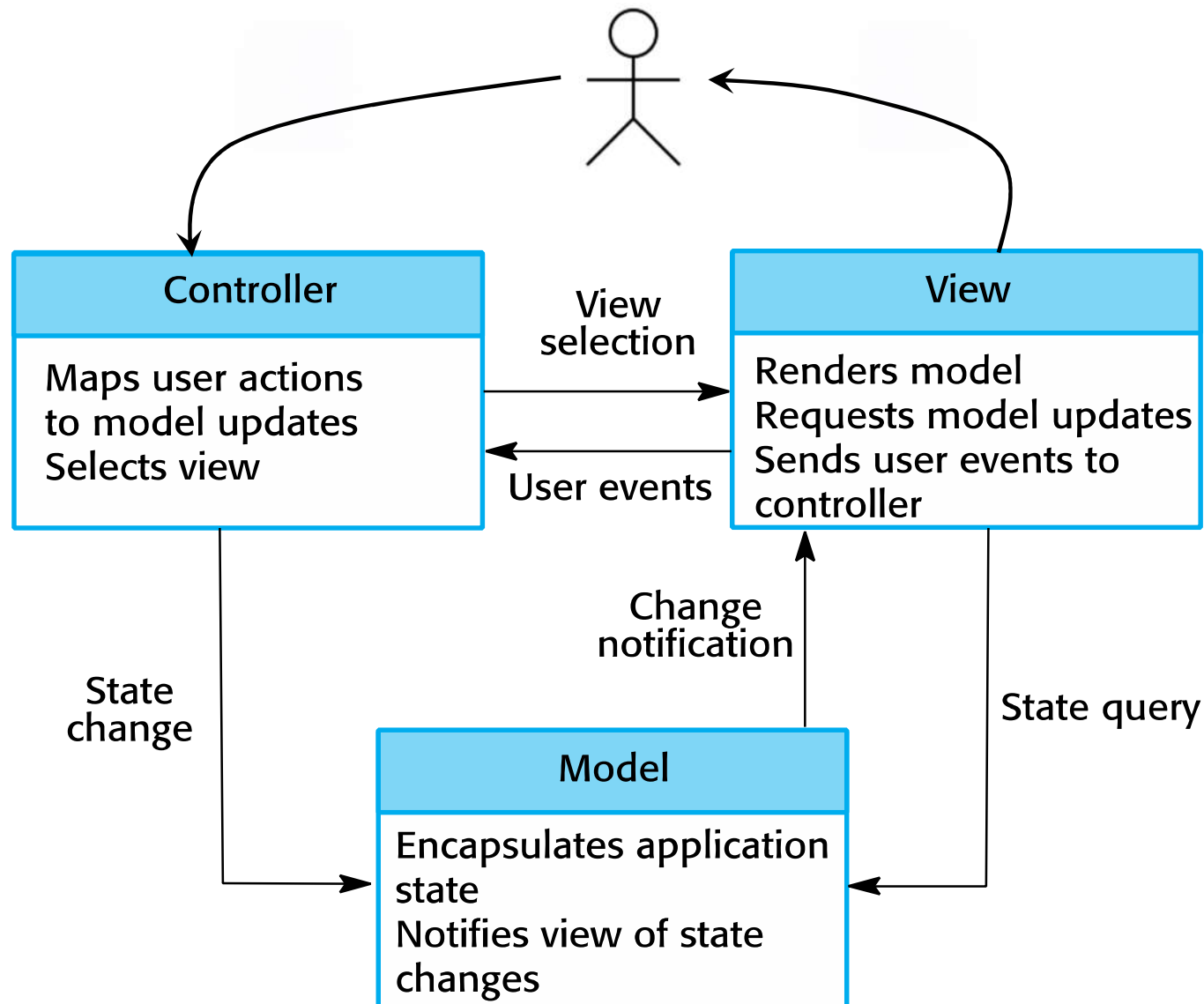
Model-View-Controller-Architektur

- Model
 - Modellierung der Datenobjekte
- View
 - Darstellung der Datenobjekte
- Controller
 - Interaktion mit den Datenobjekten, Verarbeitung und Weiterleitung von Benutzerinteraktionen
 - Viewauswahl

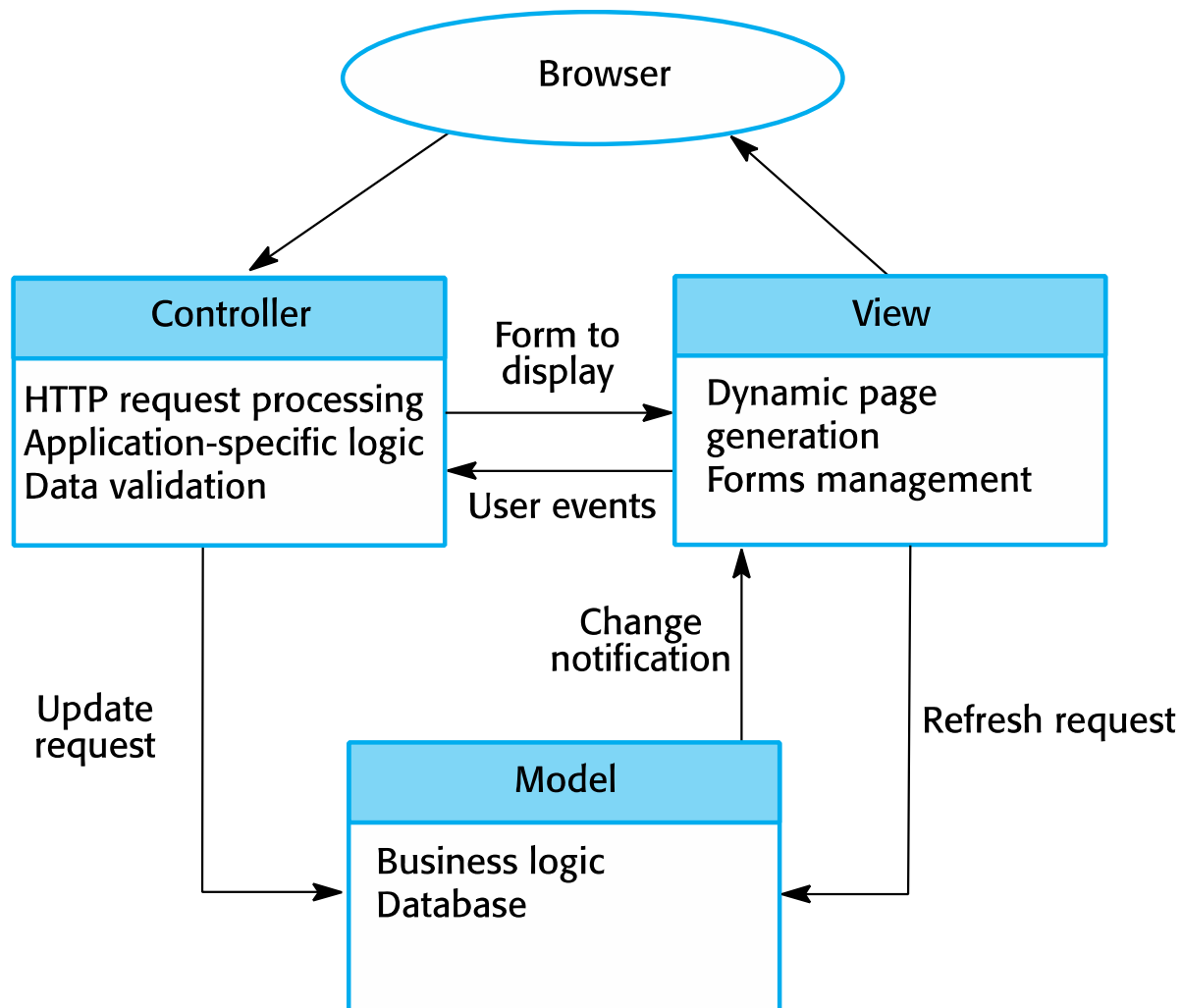


Trennt Daten, Darstellung und Interaktion voneinander:
Separation of Concerns.

Model-View-Controller, formaler



Beispiel: Webanwendungen mit MVC



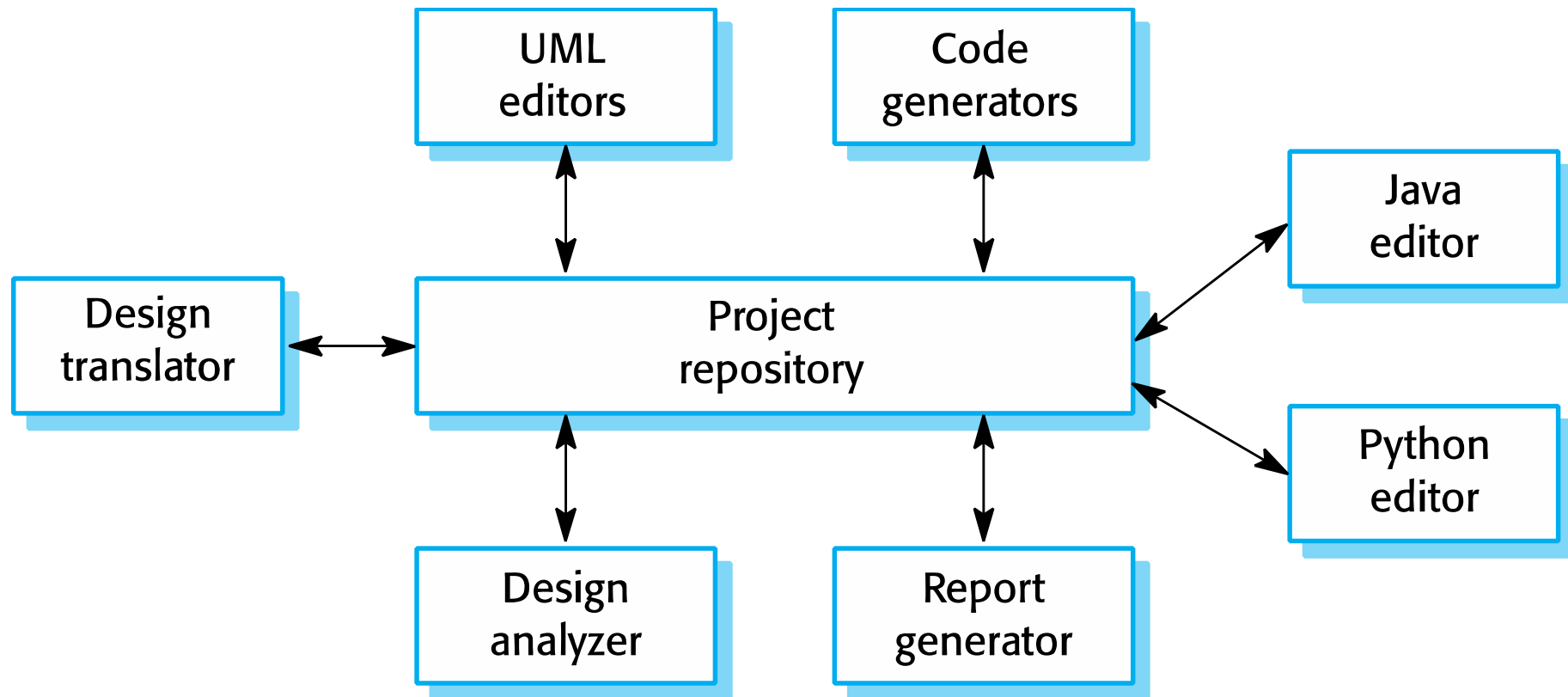
Model-View-Controller (MVC)

Name	MVC (Model-View-Controller)
Einsatz	Wenn es mehrere Wege der Darstellung und Interaktion mit Daten gibt. Dazu wenn zukünftige Anforderungen zu Interaktion und Darstellung unbekannt sind.
Vorteile	Erlaubt, dass Daten sich unabhängig von der Darstellung verändern – und umgekehrt. Unterstützt Datendarstellung auf verschiedenen Wegen. Erlaubt nebenläufige Entwicklung der Aspekte Daten – Darstellung – Interaktion, führt zu leichterer Systemerweiterbarkeit (statisch und dynamisch), verbessert insgesamt Wartbarkeit und Testbarkeit
Nachteile	Kann zu zusätzlichem Code und zusätzlicher Code-Komplexität führen, wenn Datenmodell und Interaktion sehr einfach sind. Eher ein Entwurfs- als Architekturmuster.

Repository-Architektur

- Subsysteme müssen normalerweise Daten austauschen. Dafür gibt es zwei Wege:
 - Gemeinsame Daten werden in einer zentralen Datenbank oder in einem **zentralen Repository** für alle Subsysteme gehalten, oder
 - Jedes Subsystem hat seine **jeweils eigene Datenverwaltung** und gibt Daten explizit an andere Subsysteme weiter
- Wenn es um **große Mengen gemeinsamer Daten** geht, ist das Repositorymodell (Umsetzung des ersten Weges oben) ein sehr häufiger und sehr effizienter Weg, um Daten auszutauschen.

Repository-Architektur für eine integrierte Entwicklungsumgebung



Repositorys-Architektur

Name	Repository
Einsatz	Für Systeme, in denen große Mengen an Information erzeugt werden, die für lange Zeit gespeichert werden müssen. Möglich auch für datengetriebene Systeme, bei denen die Hinzunahme von Daten im Repository eine Aktion auslöst oder ein Werkzeug startet.
Vorteile	Die Komponenten können unabhängig voneinander sein und müssen nicht voneinander wissen. Veränderungen, die eine Komponente durchführt, können an alle Komponenten weitergeleitet (propagiert) werden. Die gesamten Daten können konsistent verwaltet werden (z.B. Backups zur selben Zeit), da alles an einem Platz ist.
Nachteile	Das Repository ist kritisch für Ausfälle (single point of failure), Probleme damit betreffen das ganze System. Eventuell Ineffizienzen bei Organisation der Kommunikation zum gesamten Repository. Verteilung des Repositoriums über mehrere Computer kann kompliziert werden.

Pipes-und-Filter-Architektur 1

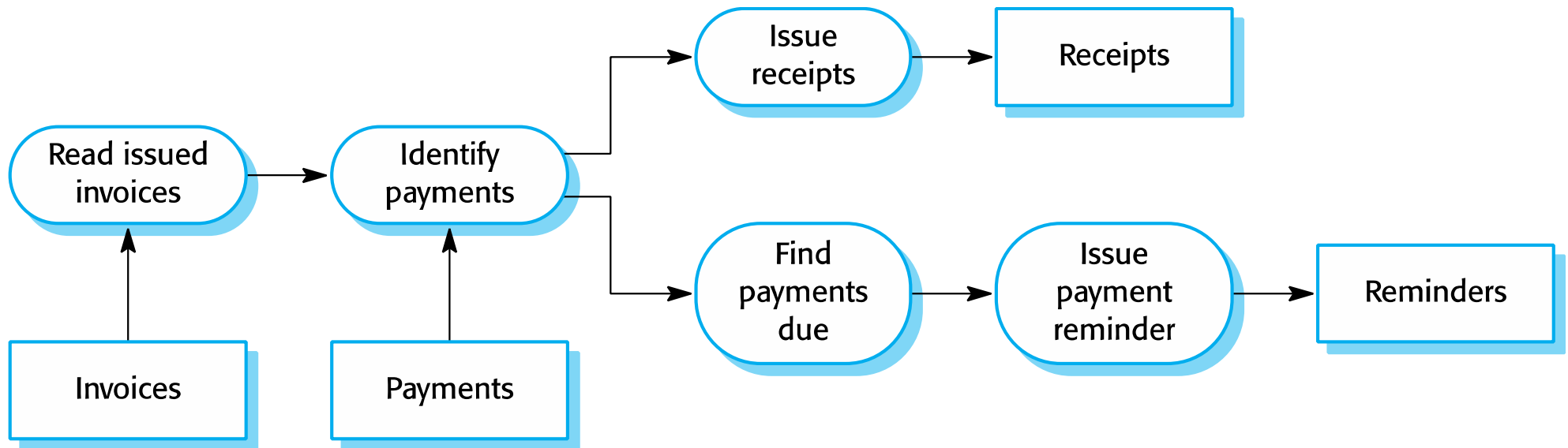
- **Situation:** Eingaben werden **funktionalen Transformationen** unterworfen um Ausgaben zu produzieren
- Bekannt etwa aus der UNIX-Shell (Verkettung von Befehlen / Prozessen)
- Die Verarbeitung der Daten in einem System wird so organisiert, dass jede **Verarbeitungskomponente** getrennt eine Art von Datentransformation ausführt.
- Dabei **fließen** die Daten (wie durch ein Rohr = Pipe) von einer Verarbeitungskomponente zur nächsten.
- Auch genannt: Datenfluss-Architektur

Pipes-und-Filter-Architektur 2

- Es gibt einige **Varianten** dieses Modells. Bei sequentiellen Transformationen erhalten wir ein **Stapelverarbeitungsmodell**, dass in Datenverarbeitungssystemen oft verwendet wird
- Kaum geeignet für interaktive Systeme

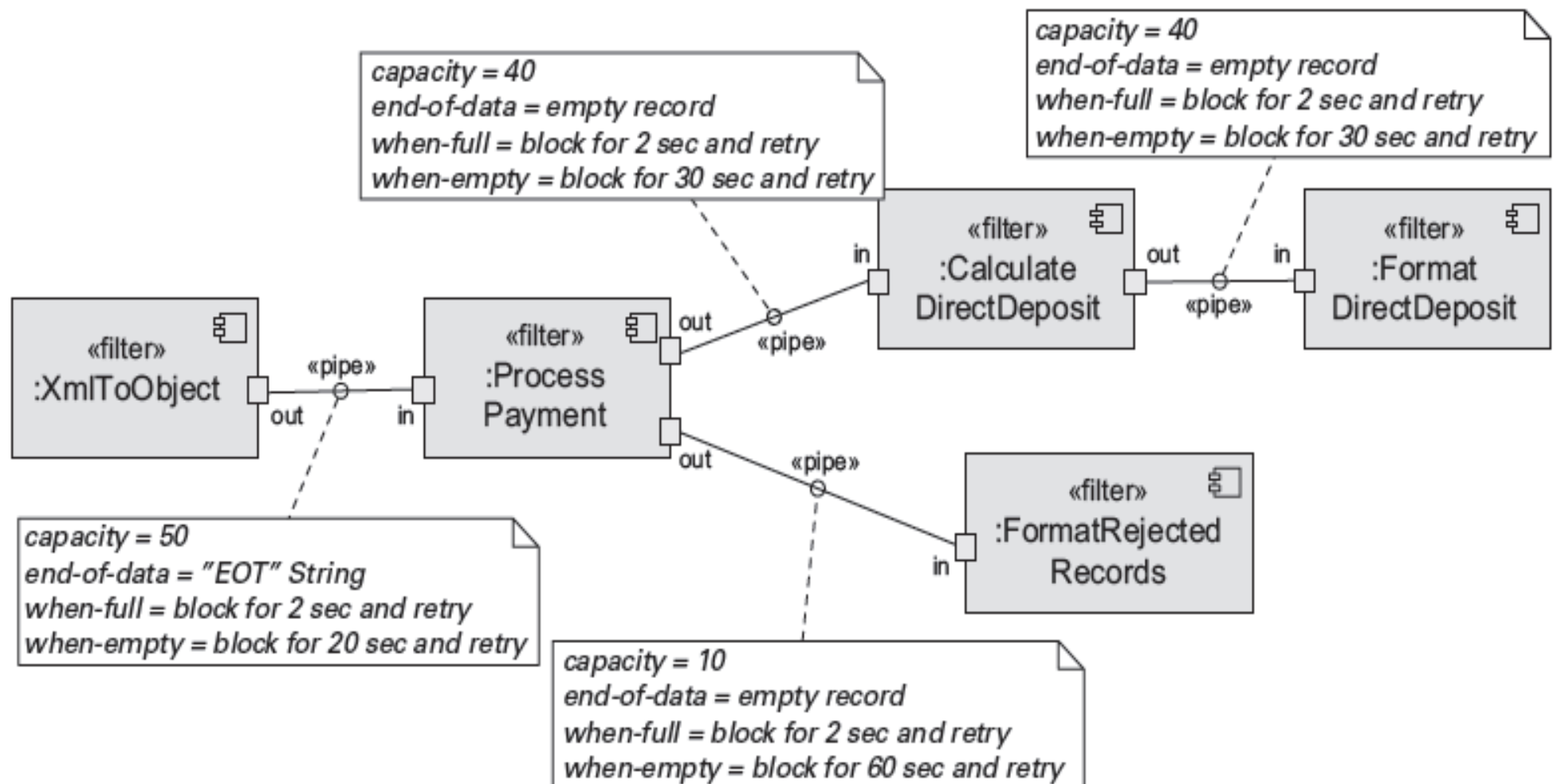
Pipes-und-Filter-Architektur

Beispiel einer Buchhaltungssoftware



Pipes-und-Filter-Architektur

Beispiel mit Spezifikation der Pipes



Bass, Len; Clements, Paul; Kazman, Rick (2013): Software Architecture in Practice. 3. Aufl. Upper Saddle River, NJ: Addison-Wesley (SEI Series in Software Engineering).

Pipes-und-Filter-Architektur

Name	Pipes-und-Filter
Einsatz	Oft verwendet bei datenverarbeitenden Anwendungen (sowohl bei Stapelverarbeitung wie auch Transaktionsbasierung), bei denen Eingaben in mehreren Stufen verarbeitet werden um Ausgaben zu produzieren.
Vorteile	Leicht zu verstehen, unterstützt Wiederverwendung von Transformationen. Der Workflow-Stil paßt zur Struktur vieler Geschäftsprozesse. Weiterentwicklung durch Hinzufügung von Transformationen ist leicht. Kann entweder als sequentielles oder paralleles System implementiert werden.
Nachteile	Das Datenformat zwischen kommunizierenden Transformationen muss festgelegt sein. Jede Transformation muss Eingaben parsen und Ausgaben in vereinbarter Form erstellen; dadurch kann es zu größerem Overhead kommen. Wiederverwendung bedingt kompatible Datenstrukturen.

Architekturmuster

Wir haben Muster mit unterschiedlichen Schwerpunkten kennengelernt:

1. Client-Server-Architektur
2. Mehrschichten-Architektur
3. Model-View-Controller (MVC)-Architektur
4. Repositoriums-Architektur
5. Pipes-und-Filter-Architektur (Datenfluss-Architektur)

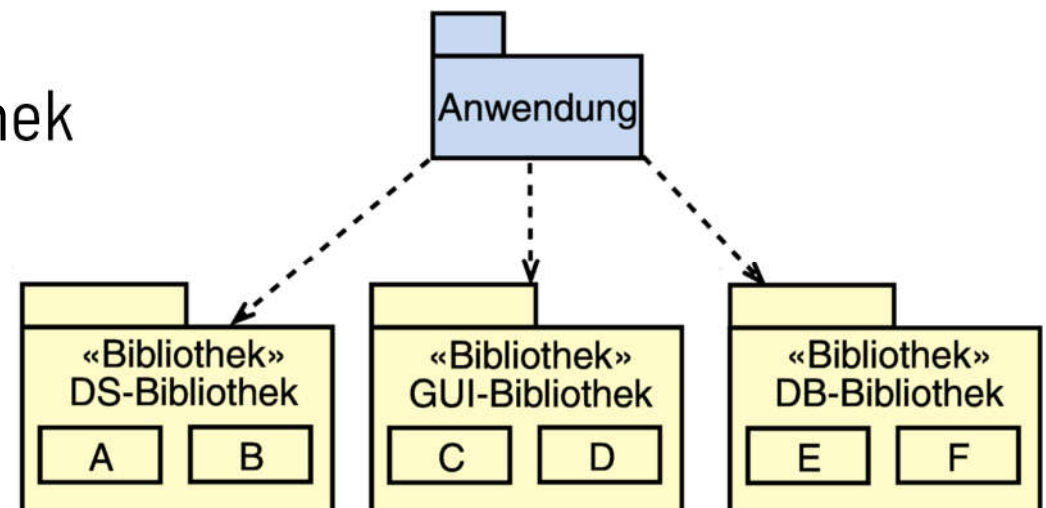
Es gibt weitere Muster: Z.B. Broker, Peer-to-Peer, Service-Orientierte Architektur mit und ohne ESB, Microservices, Map-Reduce, Multi-Tier, Event-driven, ...

Bibliotheken

Bereits in den Sechzigerjahren wurden **Programmbibliotheken** (z.B. für Matrixoperationen in Fortran) entwickelt und eingesetzt.

*Eine **Klassenbibliothek** besteht aus einer Menge von Klassen, die wiederverwendbar sind und allgemein – also unabhängig vom Anwendungskontext – nutzbare Funktionalität anbieten.*

Aus Sicht der Anwendung werden die Klassen einer Bibliothek **direkt benutzt**, oder die Klassen der Anwendung **erben** von den Bibliotheksklassen.



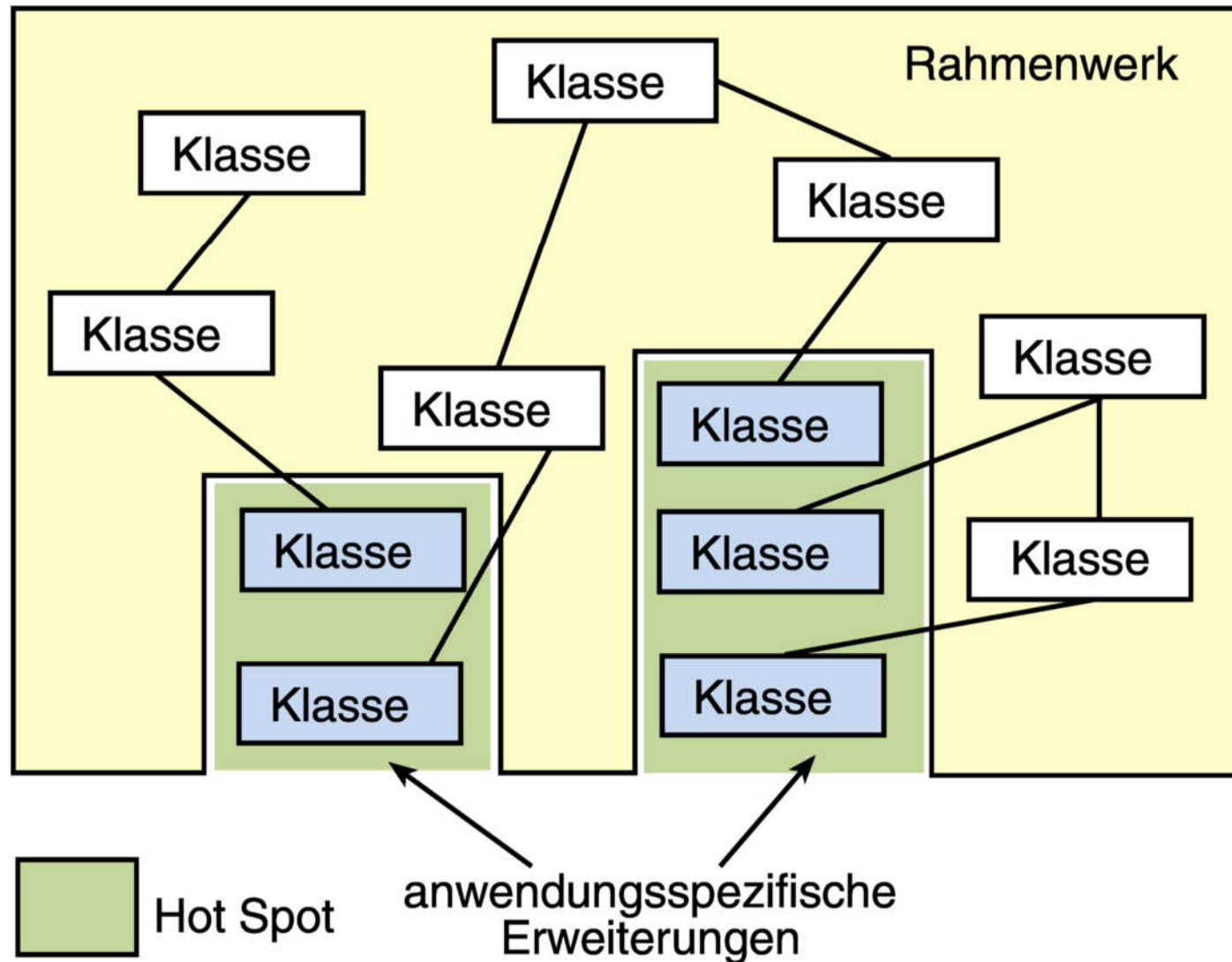
Rahmenwerke

*Ein **Rahmenwerk** (Framework) ist eine Architektur aus Klassenhierarchien, die eine allgemeine generische Lösung für ähnliche Probleme in einem bestimmten Kontext vorgibt.*

Züllighoven (2005)

- Wenn immer wieder sehr ähnliche Anwendungen entwickelt werden, sollten die Anwendungen auf der Basis einer **generischen Lösung** entwickelt werden.
- Ein Rahmenwerk hat **definierte Schnittstellen**, an denen die generische Lösung durch **anwendungsspezifischen Code** erweitert werden kann.
- Diese Stellen werden als **Hot Spots** bezeichnet.

Schema eines Rahmenwerks



Zusammenfassung

- Verschiedene Architekturmuster bieten für eine Reihe von Einsatzzwecke Lösungsmuster
- Diese Muster sind miteinander kombinierbar
- Es gibt noch mehr Architekturmuster oder Varianten! Recherche vor Entwurf einer Architektur kann sich deshalb lohnen.

